1-

| Label | Design Requirement Description | Stimulus Generation | Functional Coverage | Functionality Check |
|---|---|---|---|---|
| ALU_1 | When the reset is asserted, the outputs should be low | Start simulation with directed reset, then randomize with reset mostly OFF | - | A checker in the testbench to make sure the output is correct |
| ALU_2 | operand2 should frequently hit 0, MAXNEG, MAXPOS | Weighted randomization on operand2 | bins: zero, max_neg, max_pos | A checker in the testbench to make sure the output is correct |
| ALU_3 | Output depends on opcode when reset=0 | No constraints | bins for ADD, SUB, transition bin (ADD => SUB), illegal bin for DIV | A checker in the testbench to make sure the output is correct |
| ALU_4 | Verify arithmetic corner cases across opcode operations | Randomize opcode with focus on ADD/SUB and drive operand1 mostly to MAXPOS, MAXNEG, and ZERO | Create cross coverage between opcode ADD/SUB and operand1 corner bins (MAXPOS, MAXNEG). Disable automatic cross bins and define only targeted bins: maxpos_add_or_sub, maxneg_add_or_sub, | A checker in the testbench to make sure the output is correct |

```
import testing_pkg::*;

module alu_seq(operand1, operand2, clk, rst, opcode, out);
input byte operand1, operand2;
input clk, rst;
input opcode_e opcode;
output byte out;

always @(posedge clk) begin
    if (rst)
        out <= 0;
    else
        case (opcode)
            ADD: out <= operand1 + operand2;
            SUB: out <= operand1 - operand2;
            MULT:out <= operand1 * operand2;
            DIV: out <= (operand2 != 0) ? operand1 / operand2 : 0;
            default: out <= 0;
        endcase
end
endmodule
```

```
package testing_pkg;

    typedef enum {ADD, SUB, MULT, DIV} opcode_e;
    parameter byte MAX_NEG = -128;
    parameter byte MAX_POS = 127;
    parameter byte ZERO    = 0;

    class transaction;
        rand opcode_e opcode;
        rand byte operand1;
        rand byte operand2;
        rand bit rst;
        bit clk;

        constraint rst_c{
            rst dist {1 :/ 4, 0 :/ 96};
        }

        constraint operand2_c{
            operand1 dist {ZERO := 20, MAX_POS := 35, MAX_NEG := 35, [-127:126] := 10};
        }

        covergroup covCode @(posedge clk);

            opcode_cp : coverpoint opcode{
                bins add_sub = {ADD, SUB};
                bins add_then_sub = (ADD => SUB);
                illegal_bins no_div = {DIV};
            }

            operand1_cp : coverpoint operand1{
                bins max_neg = {MAX_NEG};
                bins max_pos = {MAX_POS};
                bins zero = {ZERO};
                bins misc = default;
            }
```

```
            opcode_operand1 : cross opcode_cp, operand1_cp{
                option.weight = 5;
                option.cross_auto_bin_max = 0;
                bins maxpos_add_or_sub = binsof(opcode_cp.add_sub) && binsof(operand1_cp.max_pos);
                bins maxneg_add_or_sub = binsof(opcode_cp.add_sub) && binsof(operand1_cp.max_neg);
            }

        endgroup

        function new();
            covCode = new();
        endfunction

    endclass


endpackage
```

```
import testing_pkg::*;

module alu_tb();

    transaction alu_obj;

    // Declare local input and outputs
    byte operand1, operand2;
    bit clk, rst;
    opcode_e opcode;
    byte out;
    byte out_exp;

    int correct_count, error_count;

    // instantiate the ALU
    alu_seq uut (
        operand1,
        operand2,
        clk,
        rst,
        opcode,
        out);

    // Clock generation
    initial begin
        clk = 0;
        forever begin
            #5 clk = ~clk; // 10 time units period
            alu_obj.clk = clk; // Update transaction clock
        end
    end
```

```
// Test stimulus
initial begin
    // initialization
    alu_obj = new();
    rst = 0;
    operand1 = 0;
    operand2 = 0;
    opcode = ADD;
    correct_count = 0;
    error_count = 0;

    #2;
    assert_reset();
    #2;
    assert_reset(); // Call reset twice to ensure it works consistently

    repeat(50) begin
        assert(alu_obj.randomize());
        rst = alu_obj.rst;
        operand1 = alu_obj.operand1;
        operand2 = alu_obj.operand2;
        opcode = alu_obj.opcode;
        check_result(alu_obj);
    end
    $display("correct_count =  %0d error_count = %0d", correct_count, error_count);
    $stop;
end

task assert_reset();
    rst = 1'b1;
    @(negedge clk);
    if (out !== 0) begin
        $display("%t: Reset failed: out = %0d, expected 0",$time, out);
        error_count++;
    end else begin
        correct_count++;
        $display("%t: Reset succeed: out = %0d",$time, out);
    end
    rst = 1'b0;
endtask
```

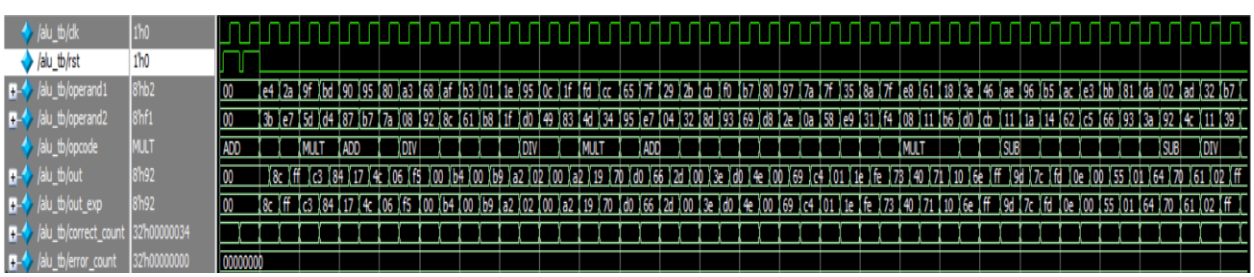
```
    task check_result(transaction alu_obj2);
        golden_model(alu_obj2);
        @(negedge clk);
        if (out == out_exp) begin
            correct_count++;
            $display("%t: out = %0d, expected = %0d",$time, out, out_exp);
        end else begin
            error_count++;
            $display("%t: out = %0d, expected = %0d",$time, out, out_exp);
        end
    endtask


    task golden_model(transaction alu_obj1);
        if(alu_obj1.rst) begin
            out_exp = 0;
        end else begin
            case (alu_obj1.opcode)
            ADD: out_exp = alu_obj1.operand1 + alu_obj1.operand2;
            SUB: out_exp = alu_obj1.operand1 - alu_obj1.operand2;
            MULT:out_exp = alu_obj1.operand1 * alu_obj1.operand2;
            DIV: out_exp = (alu_obj1.operand2 != 0) ? alu_obj1.operand1 / alu_obj1.operand2 : 0;
            default: out_exp <= 0;
        endcase
        end
    endtask

endmodule
```

```
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         5         5         0   100.00%

--------------------------------Branch Details--------------------------------
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       6         6         0   100.00%
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         56        56         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /alu_tb/uut --

                                  Node      1H->0L      0L->1H                        "Coverage"
                                  ----------------------------------------------------------------
                                   clk           1           1                            100.00
                                opcode                ENUM type       Value       Count
                                                          ADD           1      100.00
                                                          SUB           1      100.00
                                                         MULT           2      100.00
                                                          DIV           1      100.00
                          operand1[7-0]           1           1                            100.00
                          operand2[7-0]           1           1                            100.00
                               out[7-0]           1           1                            100.00
                                   rst           1           1                            100.00
```

| | | | |
|---|---|---|---|
| /testing_pkg/transaction | 100.00% | | |
| TYPE covCode | 100.00% | 100 | 100.00% |
| CVP covCode::opcode_cp | 100.00% | 100 | 100.00% |
| CVP covCode::operand1_cp | 100.00% | 100 | 100.00% |
| CROSS covCode::opcode_operand1 | 100.00% | 100 | 100.00% |
| INST \/testing_pkg::transaction::covCode | 100.00% | 100 | 100.00% |
| CVP opcode_cp | 100.00% | 100 | 100.00% |
| illegal_bin no_div | 15 | - | - |
| bin add_sub | 22 | 1 | 100.00% |
| bin add_then_sub | 1 | 1 | 100.00% |
| CVP operand1_cp | 100.00% | 100 | 100.00% |
| bin max_neg | 2 | 1 | 100.00% |
| bin max_pos | 3 | 1 | 100.00% |
| bin zero | 2 | 1 | 100.00% |
| default bin misc | 45 | - | - |
| CROSS opcode_operand1 | 100.00% | 100 | 100.00% |
| bin maxpos_add_or_sub | 2 | 1 | 100.00% |
| bin maxneg_add_or_sub | 1 | 1 | 100.00% |



2-

| Label | Design Requirement Description | Stimulus Generation | Functional Coverage | Functionality Check |
|---|---|---|---|---|
| COUNTER_1 | When the reset is asserted, the output counter value should be low | Directed at the start of the simulation | - | A checker in the testbench to make sure the output is correct |
| COUNTER_2 | When the load is asserted, the output count_out should take the value of the load_data input | Randomization under Constraint the load signal to be active 70% of the time | Cover All values of load data | A checker in the testbench to make sure the output is correct |
| COUNTER_3 | When the load/reset is deasserted, the output count_out increment as long as the enable is active and up_down is high | Randomization for up_down and Randomization under Constraint the enable signal to be active 70% of the time | Cover All values of count_out and transition bin from max to zero | A checker in the testbench to make sure the output is correct |
| COUNTER_4 | When the load/reset is deasserted, the output count_out decrement as long as the enable is active and up_down is low | Randomization for up_down and Randomization under Constraint the enable signal to be active 70% of the time | Cover All values of count_out and transition bin from zero to max | A checker in the testbench to make sure the output is correct |

```
////////////////////////////////////////////////////////////////////////////////
// Author: Kareem Waseem
// Course: Digital Verification using SV & UVM
//
// Description: Counter Design
//
////////////////////////////////////////////////////////////////////////////////
module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);
parameter WIDTH = 4;
input clk;
input rst_n;
input load_n;
input up_down;
input ce;
input [WIDTH-1:0] data_load;
output reg [WIDTH-1:0] count_out;
output max_count;
output zero;

always @(posedge clk) begin
    if (!rst_n)
        count_out <= 0;
    else if (!load_n)
        count_out <= data_load;
    else if (ce)
        if (up_down)
            count_out <= count_out + 1;
        else
            count_out <= count_out - 1;
end

assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
assign zero = (count_out == 0)? 1:0;

endmodule
```

```
package counter_pkg;

    class counter_class;

        parameter COUNTER_WIDTH = 4;
        rand bit rst_n, load_n, up_down, ce;
        rand bit [COUNTER_WIDTH-1:0] data_load;
        bit clk;
        bit [COUNTER_WIDTH-1:0] count_out;

        constraint rst_cst{
            rst_n dist {0:= 5, 1:= 95};
        }

        constraint load_cst{
            load_n dist{0:= 70, 1:= 30};
        }

        constraint ce_cst{
            ce dist{0:= 30, 1:= 70};
        }

        constraint up_down_cst{
            up_down dist{0:= 50, 1:= 50};
        }
```

```
        covergroup counter_cvg @(posedge clk);

            // 1) Load data coverage
            load_cvg: coverpoint data_load iff (rst_n && !load_n);

            // 2) Count UP all values
            count_up_cvg: coverpoint count_out
                iff (rst_n && ce && up_down){
                    bins all_vals[] = {[0:(2**COUNTER_WIDTH) -1]};
                }

            // 3) Overflow transition
            overflow_cfg: coverpoint count_out
                iff (rst_n && ce && up_down){
                    bins overflow = ((2**COUNTER_WIDTH)-1 => 0);
                }

            // 4) Count DOWN all values
            count_down_cvg: coverpoint count_out
                iff (rst_n && ce && !up_down){
                    bins all_vals[] = {[0:(2**COUNTER_WIDTH) -1]};
                }

            // 3) Underflow transition
            underflow_cfg: coverpoint count_out
                iff (rst_n && ce && !up_down){
                    bins underflow = (0 => (2**COUNTER_WIDTH)-1);
                }

        endgroup

        function new();
            counter_cvg = new();
        endfunction

    endclass

endpackage
```

```
import counter_pkg::*;

module counter_tb();

    counter_class stimulus = new();

    //1 declare local wire and reg
    bit clk_tb, rst_n_tb, load_n_tb, up_down_tb, ce_tb;
    bit [stimulus.COUNTER_WIDTH-1:0] data_load_tb;
    logic [stimulus.COUNTER_WIDTH-1:0] count_out_tb, count_out_exp;
    logic max_count_tb, max_count_exp;
    logic zero_tb, zero_exp;

    int error_count, correct_count;

    //2 Instantiate the module under test
    counter #(.WIDTH(stimulus.COUNTER_WIDTH)) uut(
            clk_tb,
            rst_n_tb,
            load_n_tb,
            up_down_tb,
            ce_tb,
            data_load_tb,
            count_out_tb,
            max_count_tb,
            zero_tb);

    //3 General stimuli using always and initial
    initial begin
        clk_tb = 0;
        forever begin
            #5 clk_tb = ~clk_tb; // Generate a clock signal with a period of 10 time units
            stimulus.clk = clk_tb; // Update the clock in the stimulus class
        end
    end
```

```
// Stimulus test
initial begin
    assert_reset();
    repeat(1000) begin
        assert (stimulus.randomize());
        rst_n_tb = stimulus.rst_n;
        load_n_tb = stimulus.load_n;
        ce_tb = stimulus.ce;
        up_down_tb = stimulus.up_down;
        data_load_tb = stimulus.data_load;
        check_result(stimulus);
    end
    repeat(500) begin
        assert (stimulus.randomize() with {load_n == 1; up_down == 1; ce == 1; rst_n == 1;});
        rst_n_tb = stimulus.rst_n;
        load_n_tb = stimulus.load_n;
        ce_tb = stimulus.ce;
        up_down_tb = stimulus.up_down;
        data_load_tb = stimulus.data_load;
        check_result(stimulus);
    end

    repeat(500) begin
        assert (stimulus.randomize() with {load_n == 1; up_down == 0; ce == 1; rst_n == 1;});
        rst_n_tb = stimulus.rst_n;
        load_n_tb = stimulus.load_n;
        ce_tb = stimulus.ce;
        up_down_tb = stimulus.up_down;
        data_load_tb = stimulus.data_load;
        check_result(stimulus);
    end

    $display("Total Correct: %0d, Total Errors: %0d", correct_count, error_count);
    $stop;
end

// Task to assert reset
task assert_reset;
    rst_n_tb = 1'b0;
    check_reset;
    rst_n_tb = 1'b1;
endtask
```

```
// Task to check reset behavior
task check_reset;
    @(negedge clk_tb);
    if ((count_out_tb !== 0) || (zero_tb !== 1) || (max_count_tb !== 0)) begin
        error_count += 1;
        $display("%t: ERROR for asserting reset--> count_out_tb = %0d, zero_tb = %0d, max_count_tb = %0d",
                $time, count_out_tb, zero_tb, max_count_tb);
    end else begin
        correct_count += 1;
        $display("%t: SUCCESS for asserting reset--> count_out_tb = %0d, zero_tb = %0d, max_count_tb = %0d",
                $time, count_out_tb, zero_tb, max_count_tb);

    end
endtask

// Task to implement the golden model and check results
task golden_model(input counter_class stimulus_c);
    if (!stimulus_c.rst_n)
        count_out_exp = 0;
    else if (!stimulus_c.load_n)
        count_out_exp = data_load_tb;
    else if (stimulus_c.ce)
        if (stimulus_c.up_down)
            count_out_exp = count_out_exp + 1;
        else
            count_out_exp = count_out_exp - 1;
    max_count_exp = (count_out_exp == {stimulus_c.COUNTER_WIDTH{1'b1}})? 1:0;
    zero_exp = (count_out_exp == 0)? 1:0;
endtask
```

```
// Task to check results against the golden model
task check_result(input counter_class stimulus_t);
    golden_model(stimulus_t);
    @(negedge clk_tb);
    stimulus.count_out = count_out_tb; // Update the count_out in the stimulus class for coverage
    if(stimulus_t.rst_n !== 0) begin
        if ((count_out_tb !== count_out_exp) || (zero_tb !== zero_exp) || (max_count_tb !== max_count_exp)) begin
        error_count += 1;
        $display("%t: ERROR--> count_out_tb = %0d,count_out_exp = %0d, zero_tb = %0d, zero_exp = %0d, max_count_tb = %0d,max_count_exp = %0d",
                $time,count_out_tb, count_out_exp, zero_tb, zero_exp, max_count_tb, max_count_exp);
        end
        else begin
            correct_count += 1;
            $display("%t: SUCCESS--> count_out_tb = %0d,count_out_exp = %0d, zero_tb = %0d, zero_exp = %0d, max_count_tb = %0d,max_count_exp = %0d",
                    $time,count_out_tb, count_out_exp, zero_tb, zero_exp, max_count_tb, max_count_exp);
        end
    end else begin
        check_reset;
    end
endtask

endmodule
```

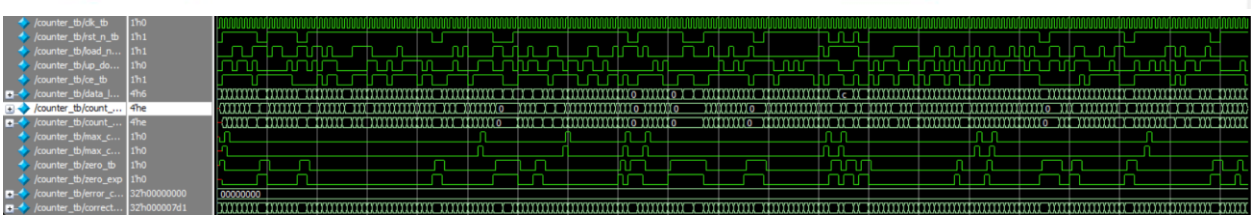
```
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
acks\Digial_IC\09-Verification\Lab3_1\Counter\counter_tb.ucdb • Untracked
    Branches                        10        10         0   100.00%

================================Branch Details================================
```

```
Condition Coverage:
    Enabled Coverage              Bins    Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Conditions                       2         2         0   100.00%

================================Condition Details================================
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       7         7         0   100.00%

================================Statement Details================================
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         30        30         0   100.00%

================================Toggle Details================================
```

| Name | Class Type | Coverage | Goal | % of Goal | Status | Included | Merge_instances | Get_inst_coverage | Cc |
|---|---|---|---|---|---|---|---|---|---|
| /counter_pkg/counter_class | | 100.00% | | | | | | | |
| TYPE counter_cvg | | 100.00% | 100 | 100.00% | | | auto(0) | | |
| CVP counter_cvg::load_cvg | | 100.00% | 100 | 100.00% | | | | | |
| CVP counter_cvg::count_up_cvg | | 100.00% | 100 | 100.00% | | | | | |
| CVP counter_cvg::overflow_cfg | | 100.00% | 100 | 100.00% | | | | | |
| CVP counter_cvg::count_down_cvg | | 100.00% | 100 | 100.00% | | | | | |
| CVP counter_cvg::underflow_cfg | | 100.00% | 100 | 100.00% | | | | | |
| INST \/counter_pkg::counter_class::counter_cvg | | 100.00% | 100 | 100.00% | | | | 0 | |
| CVP load_cvg | | 100.00% | 100 | 100.00% | | | | | |
| CVP count_up_cvg | | 100.00% | 100 | 100.00% | | | | | |
| CVP overflow_cfg | | 100.00% | 100 | 100.00% | | | | | |
| CVP count_down_cvg | | 100.00% | 100 | 100.00% | | | | | |
| CVP underflow_cfg | | 100.00% | 100 | 100.00% | | | | | |



3-

| Label | Design Requirement Description | Stimulus Generation | Functional Coverage | Functionality Check |
|---|---|---|---|---|
| ALSU_1 | When the reset is asserted, the outputs should be low | Directed at the start of the simulation | bins: out_low | A checker in the testbench to make sure the output is correct |
| ALSU_2 | Incase of invalid cases occur, Leds should blink and out bits are set to low, | Randomization under constraint where invalid cases shouldn't occur as frequent as valid cases | bins: Leds_blink, out_low, bins_invalid | A checker in the testbench to make sure the output is correct |
| ALSU_3 | Incase of invalid cases do not occur,a and the bypass_A or bypass_B are high then the output will take the value of A or B depending on the parameter INPUT_PRIORITY | Randomization under constraint on bypass ports are disabled most of the time | bins: A_data_0, A_data_max, A_data_min, A_data_default,B_data_0, B_data_max, B_data_min, B_data_default | A checker in the testbench to make sure the output is correct |
| ALSU_4 | Incase of invalid cases do not occur, when opcode is ADD, then out should perform the addition on ports A and B taking cin if parameter FULL_ADDER is high | Randomization Constraint for adder inputs (A, B) to take the values (MAXPOS, ZERO and MAXNEG) more often than the other values | bins: A_data_0, A_data_max, A_data_min, A_data_default,B_data_0, B_data_max, B_data_min, B_data_default, bins_arith | A checker in the testbench to make sure the output is correct |
| ALSU_5 | Incase of invalid cases do not occur, when opcode is MULT, then out should perform the multiplication on ports A and B | Randomization Constraint for adder inputs (A, B) to take the values (MAXPOS, ZERO and MAXNEG) more often than the other values | bins: A_data_0, A_data_max, A_data_min, A_data_default,B_data_0, B_data_max, B_data_min, B_data_default, bins_arith | A checker in the testbench to make sure the output is correct |
| ALSU_6 | Incase of invalid cases do not occur, when opcode is OR, then out should perform the OR operation on ports A and B if reduction_A and reduction_B are low | Randomization without constraint on A or B. | bins: A_data_0, A_data_max, A_data_min, A_data_default,B_data_0, B_data_max, B_data_min, B_data_default, | A checker in the testbench to make sure the output is correct |
| ALSU_7 | Incase of invalid cases do not occur, when opcode is OR and if reduction_A or reduction_B is high, then output should perform the reduction OR operation on ports A or ports B depends on the priority | Randomization uunder the constraint on the input A most of the time to have one bit high in its 3 bits while constraining the B bits to be low if reduction_A is high | bins: A_data_walkingones[] If only the red_op_A is high, B_data_walkingones[] If only the red_op_B is high, , bins_bitwise | A checker in the testbench to make sure the output is correct |
| ALSU_8 | Incase of invalid cases do not occur, when opcode is OR, then out should perform the OR operation on ports A and B if reduction_A and reduction_B are low | Randomization without constraint on A or B. | bins: A_data_0, A_data_max, A_data_min, A_data_default,B_data_0, B_data_max, B_data_min, B_data_default, , bins_bitwise | A checker in the testbench to make sure the output is correct |
| ALSU_9 | Incase of invalid cases do not occur, when opcode is XOR and if reduction_A or reduction_B is high, then output should perform the reduction XOR operation on ports A or ports B depends on the priority | Randomization uunder the constraint on the input A most of the time to have one bit high in its 3 bits while constraining the B bits to be low if reduction_A is high | bins: A_data_walkingones[] If only the red_op_A is high, B_data_walkingones[] If only the red_op_B is high, bins_bitwise | A checker in the testbench to make sure the output is correct |
| ALSU_10 | Incase of invalid cases do not occur, when opcode is SHIFT, then out should be shifted by 1 bit depending on direction bit | Randomization without constraint | bins: Bins_shift | A checker in the testbench to make sure the output is correct |
| ALSU_11 | Incase of invalid cases do not occur, when opcode is ROTATE, then out should be rotated by 1 bit depending on direction bit | Randomization without constraint | bins: Bins_shift | A checker in the testbench to make sure the output is correct |
| ALSU_12 | Incase of invalid cases do not occur, watch the Transition from opcode 0 > 1 > 2 > 3 > 4 > 5 | | bins: bins_trans | |

```
package ALSU_pkg;

    typedef enum bit [2:0] { OR, XOR, ADD, MULT, SHIFT, ROTATE, INVALID6, INVALID7 } opcode_e;
    typedef enum bit [2:0] { MAXPOS = 3'b011, ZERO = 3'b000, MAXNEG = 3'b100 } reg_e;

    class ALSU_cst;
        // Declare the random variables
        rand logic cin, rst, red_op_A, red_op_B;
        rand logic bypass_A, bypass_B, direction, serial_in;

        // Declare the random variable for opcode and an array of opcodes
        rand opcode_e opcode;
        rand opcode_e opcode_arr [0:5];

        // Declare the random variables for inputs A and B, and their corner cases and walking ones patterns
        rand logic [2:0] A, B;
        rand reg_e enum_ext_A, enum_ext_B;
        rand logic [2:0] A_rem_val, B_rem_val;

        bit [2:0] walking_ones[] = '{3'b100, 3'b010, 3'b001};
        rand bit [2:0] walking_ones_t, walking_ones_f;

        // Declare the clock signal for the covergroup
        bit clk;

        //1. Reset to be asserted with a low probability that you decide.
        constraint rst_cst{
            rst dist{0:= 99, 1:= 1};
        }
```

```
        //2. The inputs A and B to be mostly zero, with some random values.
        constraint A_B_cst{
            walking_ones_t inside{walking_ones}; // Constrain the walking_ones_t to be one of the patterns in walking_ones
            !(walking_ones_f inside {walking_ones}); // Constrain the walking_ones_f to not be any of the patterns in walking_ones
            !(A_rem_val inside {enum_ext_A}); // Constrain A_rem_val to not be any of the corner cases
            !(B_rem_val inside {enum_ext_B}); // Constrain B_rem_val to not be any of the corner cases

            // If the opcode is OR or XOR
            if ((opcode == OR) || (opcode == XOR)){
                // red_op_A is high, constraint the input A most of the time
                // to have one bit high in its 3 bits while constraining the B bits to be low
                if (red_op_A == 1){
                    A dist {walking_ones_t := 90, walking_ones_f := 10};
                    B == 3'b000;
                }
                // red_op_B is high, constraint the input B most of the time
                // to have one bit high in its 3 bits while constraining the A bits to be low
                else if (red_op_B == 1){
                    B dist {walking_ones_t := 90, walking_ones_f := 10};
                    A == 3'b000;
                }
            } else {
                red_op_A dist {1:= 10, 0 := 90};
                red_op_B dist {1:= 10, 0 := 90};
                // If the opcode is ADD or MULT, constraint the inputs A and B  to take the values (MAXPOS, ZERO and MAXNEG) with higher probability than the other value
                if ((opcode == ADD) || (opcode == MULT)){
                A dist {enum_ext_A := 90, A_rem_val := 10};
                B dist {enum_ext_B := 90, B_rem_val := 10};
            }
            }
        }
```

```
// 3. The opcode to be mostly OR, XOR, ADD, MULT, ROTATE, SHIFT, with some random values for the other operations.
// will be used while turning off opcode_arr_cst to allow random values in the opcode_arr which will be used in the covergroup to cover the transitions between all the operations.
constraint opcode_cst{
    opcode dist{[OR:ROTATE] := 90, [INVALID6:INVALID7] := 10};
}

// 4. The bypass signals to be mostly disabled, with some random values.
constraint bypass_cst{
    bypass_A dist{0:= 95, 1:= 5};
    bypass_B dist{0:= 95, 1:= 5};
}

// Create a fixed array of 6 elements of type opcode_e. constraint the elements of the array using
// foreach to have a unique valid value each time randomization occurs.
// will be used in the covergroup to cover all the operations and the transitions between them while turning off opcode_cst to allow the array to be randomized with values from OR to ROTATE.
constraint opcode_arr_cst{

    foreach(opcode_arr[i]){
        opcode_arr[i] inside {[OR:ROTATE]};
    }
    unique{opcode_arr};

}
```

```
    covergroup cvr_gp @(posedge clk);

        // Cover A values (corner cases + misc)
        A_cp1: coverpoint A {
            bins A_data_0 = {3'b000};
            bins A_data_max = {3'b011};
            bins A_data_min = {3'b100};
            bins A_data_default = default;
        }

        // Walking ones when red_op_A = 1
        A_cp2: coverpoint A iff(red_op_A){
            bins A_data_walkingones[] ={3'b001, 3'b010, 3'b100};
        }

        // Cover B values (corner cases + misc)
        B_cp1: coverpoint B {
            bins B_data_0 = {3'b000};
            bins B_data_max = {3'b011};
            bins B_data_min = {3'b100};
            bins B_data_default = default;
        }

        // Walking ones when red_op_B = 1
        B_cp2: coverpoint B iff(red_op_B  && !red_op_A ) {
            bins B_data_walkingones[] ={3'b001, 3'b010, 3'b100};
        }

        // Cover the opcode with the specified bins and transitions
        ALU_cp: coverpoint opcode{
            bins bins_shift[] = {SHIFT, ROTATE};
            bins bins_arith[] = {ADD, MULT};
            bins bins_bitwise[] = {OR, XOR};
            bins bins_invalid[] = {INVALID6, INVALID7};
            bins bins_trans1 = (OR => XOR );
            bins bins_trans2 = (OR => XOR => ADD );
            bins bins_trans3 = (OR => XOR => ADD => MULT );
            bins bins_trans4 = (OR => XOR => ADD => MULT => SHIFT);
            bins bins_trans5 = (OR => XOR => ADD => MULT => SHIFT => ROTATE);
        }

    endgroup
```

```
        function new();
            cvr_gp = new();
        endfunction

        // Sample task: only sample when not in reset or bypass mode
        task sample_coverage();
            if (!rst && !bypass_A && !bypass_B)
                cvr_gp.sample();
        endtask


    endclass

endpackage
```

```
import ALSU_pkg::*;

module ALSU_tb;

    // ─── 1. Parameters, classes, wires and regs ──────────────────────────

    parameter INPUT_PRIORITY = "A";
    parameter FULL_ADDER = "ON";
    bit clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
    bit signed [2:0] A, B;
    bit [2:0] opcode;
    logic [15:0] leds;
    bit [15:0] leds_exp;
    logic signed [5:0] out;
    bit signed [5:0] out_exp;

    // Registered stimulus used by the golden model
    bit red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
    bit signed [1:0] cin_reg;
    bit [2:0] opcode_reg;
    bit signed [2:0] A_reg, B_reg;


    int error_count = 0;
    int correct_count = 0;


    ALSU_cst ALSU_obj = new();
```

```
    // ─── 2. DUT instantiation ────────────────────────────────────────────
    ALSU #(.INPUT_PRIORITY(INPUT_PRIORITY), .FULL_ADDER(FULL_ADDER)) uut(
        .clk(clk),
        .cin(cin),
        .rst(rst),
        .red_op_A(red_op_A),
        .red_op_B(red_op_B),
        .bypass_A(bypass_A),
        .bypass_B(bypass_B),
        .direction(direction),
        .serial_in(serial_in),
        .opcode(opcode),
        .A(A),
        .B(B),
        .leds(leds),
        .out(out)
    );

    // ─── 3. Clock generation ─────────────────────────────────────────────
    initial begin
        clk = 1'b0;
        forever begin
            #5 clk = ~clk;
            ALSU_obj.clk = clk;
        end
    end
```

```
// ─── 4. Stimulus generation ──────────────────────────────────────────
initial begin
    #5;
    // assert reset
    assert_reset;

    // ── First Loop: constraints 1-7 active, constraint 8 disabled ──────
    $display("___________________First Loop_______________________");
    ALSU_obj.opcode_arr_cst.constraint_mode(0);

    repeat(1000) begin
        assert(ALSU_obj.randomize());
        cin = ALSU_obj.cin;
        rst = ALSU_obj.rst;
        red_op_A = ALSU_obj.red_op_A;
        red_op_B = ALSU_obj.red_op_B;
        bypass_A = ALSU_obj.bypass_A;
        bypass_B = ALSU_obj.bypass_B;
        direction = ALSU_obj.direction;
        serial_in = ALSU_obj.serial_in;
        opcode = ALSU_obj.opcode;
        A = ALSU_obj.A;
        B = ALSU_obj.B;
        if(ALSU_obj.rst) begin
            check_reset();
        end else begin
            ALSU_obj.sample_coverage();
            golden_model();
            check_result();
        end
    end

    // ── Between loops: disable all constraints, force signals to 0 ──────
    ALSU_obj.constraint_mode(0); // disable constraints 1-7
    ALSU_obj.rst.rand_mode(0);
    ALSU_obj.red_op_A.rand_mode(0);
    ALSU_obj.red_op_B.rand_mode(0);
    ALSU_obj.bypass_A.rand_mode(0);
    ALSU_obj.bypass_B.rand_mode(0);
    ALSU_obj.opcode_arr_cst.constraint_mode(1); // enable constraint 8
```

```
    // ── Second Loop: unique opcode sequence, randomise other inputs ────────
    $display("_________________Second Loop_______________________");
    repeat(10) begin
        assert(ALSU_obj.randomize());

        // Apply inputs that stay constant across the 6 inner iterations
        cin = ALSU_obj.cin;
        direction = ALSU_obj.direction;
        serial_in = ALSU_obj.serial_in;
        A = ALSU_obj.A;
        B = ALSU_obj.B;

        // Iterate over the 6 unique valid opcodes
        for (int i = 0; i < 6; i++) begin
            opcode = ALSU_obj.opcode;
            ALSU_obj.sample_coverage();
            golden_model();
            check_result();
        end
    end


    $display("Total Correct: %0d, Total Errors: %0d", correct_count, error_count);
    $stop;
end

 // ─── Assert and check reset ──────────────────────────────────────────
task assert_reset;
    rst = 1'b1;
    @(negedge clk);
    check_reset;
    rst = 1'b0;
endtask
```

```
// Task to check reset behavior
task check_reset;
    @(negedge clk);
    if ((leds !== 0) || (out !== 0)) begin
        error_count += 1;
        $display("%t: ERROR for asserting reset--> leds = %0d, out = %0d",
                $time, leds, out);
    end else begin
        correct_count += 1;
        $display("%t: SUCCESS for asserting reset--> leds = %0d, out = %0d",
                $time, leds, out);
        reset_internals();
    end
endtask

task reset_internals();
    {red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg} = 6'b00_0000;
    cin_reg = 2'b00;
    opcode_reg = 3'b000;
    {A_reg,B_reg} = 6'b000000;
    leds_exp   = 16'd0;
    out_exp    = 6'd0;
endtask

// ─── Validity check (mirrors DUT logic) ──────────────────────────────
function bit is_invalid();
    bit invalid;
    if((red_op_A_reg || red_op_B_reg) && (opcode_reg > 3'b001))
        invalid = 1'b1;
    else if (opcode_reg == INVALID6 || opcode_reg == INVALID7)
        invalid = 1'b1;
    else
        invalid = 1'b0;
    return invalid;
endfunction
```

```
// Task to implement the golden model and check results
task golden_model ();
    if (is_invalid()) begin
        leds_exp = ~leds_exp;
    end
    else begin
        leds_exp = 0;
    end

    if(bypass_A_reg)
        out_exp = A_reg;
    else if(bypass_B_reg)
        out_exp = B_reg;

    else if(is_invalid())
        out_exp = 0;

    else begin
        if(opcode_reg == OR) begin
            if(red_op_A_reg)
                out_exp = |A_reg;
            else if (red_op_B_reg)
                out_exp = |B_reg;
            else
                out_exp = A_reg | B_reg;
        end

        else if(opcode_reg == XOR) begin
            if(red_op_A_reg)
                out_exp = ^A_reg;
            else if (red_op_B_reg)
                out_exp = ^B_reg;
            else
                out_exp = A_reg ^ B_reg;
        end
```

```
            else if(opcode_reg == ADD)
                out_exp = A_reg + B_reg + cin_reg;

            else if(opcode_reg == MULT)
                out_exp = A_reg * B_reg;

            else if (opcode_reg == SHIFT) begin
                if(direction_reg)
                    out_exp = {out_exp[4:0], serial_in_reg};
                else
                    out_exp = {serial_in_reg, out_exp[5:1]};
            end

            else if (opcode_reg == ROTATE) begin
                if(direction_reg)
                    out_exp = {out_exp[4:0], out_exp[5]};
                else
                    out_exp = {out_exp[0], out_exp[5:1]};
            end
        end
        update_internals();
    endtask

    task update_internals();
        cin_reg = cin;
        red_op_A_reg = red_op_A;
        red_op_B_reg = red_op_B;
        bypass_A_reg = bypass_A;
        bypass_B_reg = bypass_B;
        direction_reg = direction;
        serial_in_reg = serial_in;
        opcode_reg = opcode;
        A_reg = A;
        B_reg = B;
    endtask
```

```
    // Task to check results against the golden model
    task check_result();
        @(negedge clk);
        if ((out_exp !== out) || (leds_exp !== leds)) begin
            error_count += 1;
            $display("%t: ERROR--> A = %0d B = %0d opcode = %0d red_op_A = %0d, red_op_B = %0d bypass_A_reg = %0d, bypass_B_reg = %0d direction_reg = %0d serial_in_reg = %0d",
                     $time, A, B, opcode, red_op_A, red_op_B, bypass_A, bypass_B,
                     direction, serial_in);
            $display("out = %0d,out_exp = %0d, leds = %0d, leds_exp = %0d",
                     out, out_exp, leds, leds_exp);

        end
        else begin
            correct_count += 1;
             $display("%t: SUCCESS--> A = %0d B = %0d opcode = %0d opcode_arr = %0p red_op_A = %0d, red_op_B = %0d bypass_A_reg = %0d, bypass_B_reg = %0d direction_reg = %0d serial_in_reg = %0d",
                     $time, A, B, opcode,ALSU_obj.opcode_arr, red_op_A, red_op_B, bypass_A, bypass_B,
                     direction, serial_in);
            $display("out = %0d,out_exp = %0d, leds = %0d, leds_exp = %0d",
                     out, out_exp, leds, leds_exp);
        end
    endtask

endmodule
```

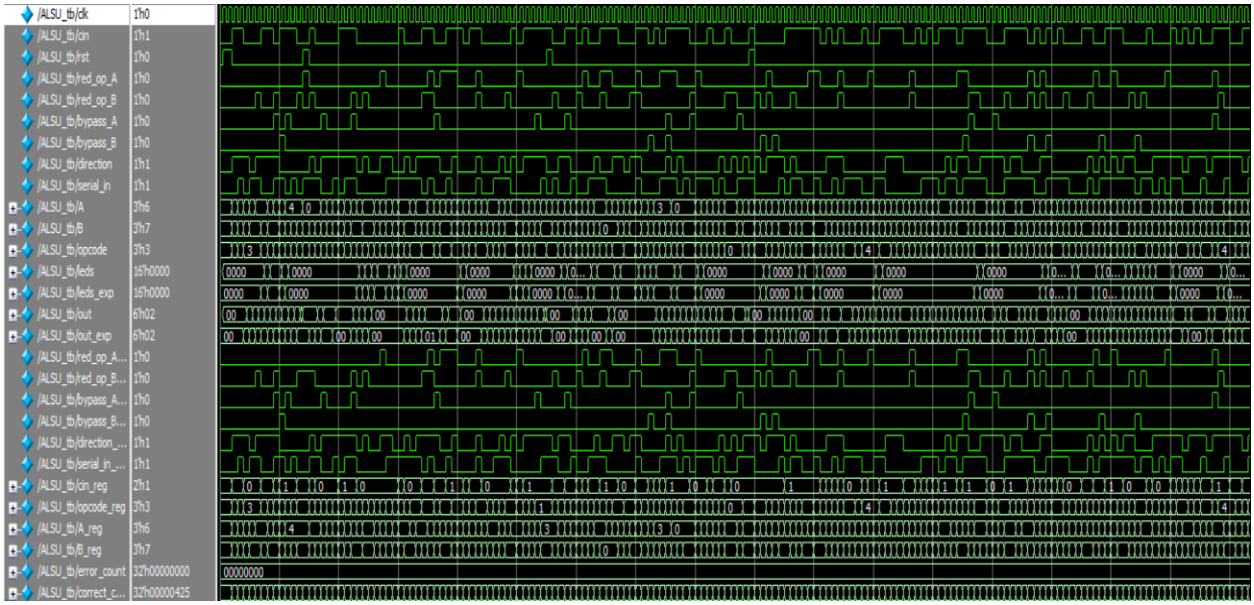
```
================================================================================
=== Instance: /ALSU_tb/uut
=== Design Unit: work.ALSU
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        31        31         0   100.00%

================================Branch Details================================
```

```
Condition Coverage:
    Enabled Coverage              Bins   Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Conditions                       6         6         0   100.00%

================================Condition Details================================
```

```
Expression Coverage:
    Enabled Coverage              Bins   Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Expressions                      8         8         0   100.00%

================================Expression Details================================
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      48        48         0   100.00%

================================Statement Details================================
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                        116       116         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /ALSU_tb/uut --
```




| | | | |
|---|---|---|---|
| **CVP** A_cp1 | 100.00% | 100 | 100.00... |
| **bin** A_data_0 | 193 | 1 | 100.00... |
| **bin** A_data_max | 174 | 1 | 100.00... |
| **bin** A_data_min | 212 | 1 | 100.00... |
| **default bin** A_data_default | 483 | - | - |
| **CVP** A_cp2 | 100.00% | 100 | 100.00... |
| **bin** A_data_walkingones[1] | 29 | 1 | 100.00... |
| **bin** A_data_walkingones[2] | 33 | 1 | 100.00... |
| **bin** A_data_walkingones[4] | 39 | 1 | 100.00... |
| **CVP** B_cp1 | 100.00% | 100 | 100.00... |
| **bin** B_data_0 | 256 | 1 | 100.00... |
| **bin** B_data_max | 174 | 1 | 100.00... |
| **bin** B_data_min | 187 | 1 | 100.00... |
| **default bin** B_data_default | 445 | - | - |
| **CVP** B_cp2 | 100.00% | 100 | 100.00... |
| **bin** B_data_walkingones[1] | 15 | 1 | 100.00... |
| **bin** B_data_walkingones[2] | 18 | 1 | 100.00... |
| **bin** B_data_walkingones[4] | 25 | 1 | 100.00... |
| **CVP** ALU_cp | 84.61% | 100 | 84.61% |
| **bin** bins_shift[SHIFT] | 184 | 1 | 100.00... |
| **bin** bins_shift[ROTATE] | 158 | 1 | 100.00... |
| **bin** bins_arith[ADD] | 156 | 1 | 100.00... |
| **bin** bins_arith[MULT] | 158 | 1 | 100.00... |
| **bin** bins_bitwise[OR] | 195 | 1 | 100.00... |
| **bin** bins_bitwise[XOR] | 153 | 1 | 100.00... |
| **bin** bins_invalid[INVALID6] | 30 | 1 | 100.00... |
| **bin** bins_invalid[INVALID7] | 28 | 1 | 100.00... |
| **bin** bins_trans1 | 33 | 1 | 100.00... |
| **bin** bins_trans2 | 4 | 1 | 100.00... |
| **bin** bins_trans3 | 1 | 1 | 100.00... |
| **bin** bins_trans4 | 0 | 1 | 0.00% |
| **bin** bins_trans5 | 0 | 1 | 0.00% |

4

```
module mymem_tb();

    //--------------------Setup and Data Structures--------------------
    localparam TESTS = 100; // number of write/read operations

    bit [15:0] address_array []; //Stores random addresses.
    bit [7:0] data_to_write_array[]; //Stores corresponding random data values.
    bit [7:0] data_read_expect_assoc [int]; //Stores expected data, indexed by address (key datatype: int)
    bit [7:0] data_read_queue[$]; //Stores the actual data read from the RAM

    // DUT port signals
    logic clk;
    logic write;
    logic read;
    logic [7:0] data_in;
    logic [15:0] address;
    logic [7:0] data_out;

    // - DUT instantiation
    my_mem uut(
        clk,
        write,
        read,
        data_in,
        address,
        data_out
    );

    // Clock Generation
    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end

    // - Counters (error_count and correct_count)
    int error_count, correct_count;
```

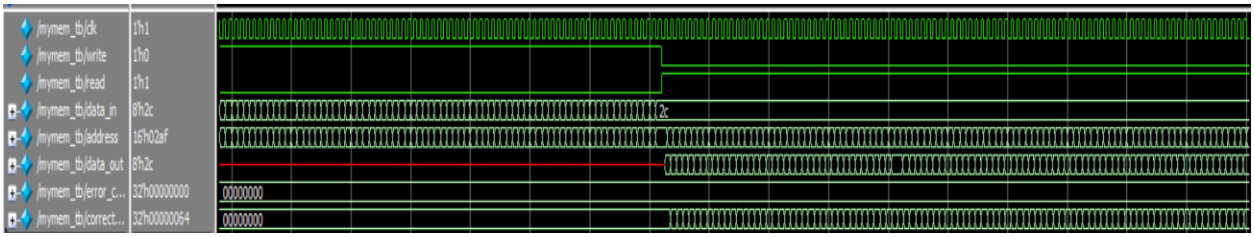
```
//---------------------Setup and Data Structures---------------------
// - Task: stimulus_gen
// - Loops TESTS times to generate:
// - Random addresses stored in address_array.
// - Random data values stored in data_to_write_array.
task stimulus_gen;
    address_array = new[TESTS];
    data_to_write_array = new[TESTS];

    for(int i = 0; i < TESTS; i++) begin
        address_array[i] = $urandom_range(0,32767); // 15-bit
        data_to_write_array[i] = $urandom_range(0,255); // 8bit
    end
endtask

// - Task: golden_model
// - Loops TESTS times to populate data_read_expect_assoc using:
// - address_array as the index.
// - data_to_write_array as expected values.
task golden_model;
    for(int i = 0; i < TESTS; i++) begin
        data_read_expect_assoc[int'(address_array[i])] = data_to_write_array[i];
    end
endtask
```

```
// - Task: check9Bits
// - Checks if the expected data out of the RAM is correct or not
task check9Bits(input logic [7:0] dut_data_out, input logic [16:0] addr);
    logic [7:0] expected;

    expected = data_read_expect_assoc[int'(addr)];
    if (dut_data_out == expected) begin
        correct_count++;
        $display("[CHECK PASS] addr=0x%02h | expected=0x%03h | got=0x%03h",
            addr, expected, dut_data_out);
    end else begin
        error_count++;
        $display("[CHECK FAIL] addr=0x%02h | expected=0x%03h | got=0x%03h",
            addr, expected, dut_data_out);
    end

endtask

//--------------------Initial block Implementation--------------------

initial begin

    // Data preparation
    data_in = 0;
    address = 0;
    read = 0;
    write = 0;

    // - Call the stimulus_gen and then the golde_model tasks to prepare the data to be sent and expected
    // data to be read from the RAM
    stimulus_gen();
    golden_model();
```

```
        // - Loop TESTS times to write data into the DUT:
        // - On each negative clock edge, drive the address and data_in ports using address_array and
        // data_to_write_array.
        @(negedge clk);
        write = 1;

        for(int i = 0; i < TESTS; i++) begin
            @(negedge clk);
            address = address_array[i];
            data_in = data_to_write_array[i];
        end

        // Read and Self-Checking
        // - Loop TESTS times to read data from the DUT:
        // - Drive the address port using address_array.
        // - Call check9Bits task to compare:
        // - Data read vs. data_read_expect_assoc[address].
        // - Store the read data into data_read_queue.
        @(negedge clk);
        write = 0;
        read = 1;
        for(int i = 0; i < TESTS; i++) begin
            @(negedge clk);
            address = address_array[i];
            @(posedge clk);
            #1;
            check9Bits(data_out,address);
            data_read_queue.push_back(data_out);
        end
```

```
        // Test Completion and Reporting
        // Print out the read data stored in data_read_queue using a while loop and pop_front method (search
        // online how to do it).
        while(data_read_queue.size() > 0) begin
            $display("queue entry = 0x%0h",data_read_queue.pop_front());
        end

        // Display error_count and correct_count
        $display("  Correct: %0d Error: %0d", correct_count, error_count);
        $stop;


    end

endmodule
```

```
=================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         3         3         0   100.00%

================================Branch Details================================
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       3         3         0   100.00%

================================Statement Details================================
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         70        67         3    95.71%

================================Toggle Details================================
```